

Technical Notes: The Model in Formulas

The companion in formulas — every pipeline step as the mathematics it implements. Reads side-by-side with HOW_THE_MODEL_WORKS.md.

Part 0. Notation and contract

Throughout, i indexes teams and R_i is team i 's Elo rating (a scalar strength, e.g. Spain 2165, Qatar 1423). A match between a designated home side H and away side A produces a scoreline $(h, a) \in \mathbb{Z}_{\geq 0}^2$. The model's prediction for a match is a scoreline $(\hat{h}, \hat{a})$. Goal rates are $\lambda_H, \lambda_A > 0$: the expected goals of each side under a Poisson model. Outcome probabilities are the triple (p_H, p_D, p_A) — home win, draw, away win — and o_j denotes a decimal bookmaker odds quote for outcome $j \in \{H, D, A\}$. The pool's score function is $S(\hat{h}, \hat{a}; h, a) \in \{0, 1, 2, 3\}$ (defined in §0). In Part II, p_t denotes the champion probability distribution over all 48 teams after conditioning on the first t played games, and d_i is team i 's learned rating drift.

Two contracts hold for the whole document:

1. **Anchor contract.** Every displayed equation is followed by an anchor naming the code that implements it — `scripts/file.py:function` — with the constants as locked before kickoff (2026-06-10). The formulas are extracted from the code, not from an idealized model.
 2. **Honesty rule.** Where the code is a pragmatic shortcut rather than clean statistics (an independence assumption, a proportional vig split, a memoized grid), the note says so explicitly rather than dressing it up.
-

Part I — The locked forecast

(Sections 0–6 mirror stages 0–6 of HOW_THE_MODEL_WORKS.md, same numbers.)

0. The goal shapes everything — the 3/2/1 objective as a decision problem

The pool awards points per match by comparing the predicted scoreline $(\hat{h}, \hat{a})$ to the realized one (h, a) . Write the realized result as $r = \text{sgn}(h - a) \in \{+1, 0, -1\}$ (home win, draw, away win) and the predicted result as $\hat{r} = \text{sgn}(\hat{h} - \hat{a})$. The score function is

$$S(\hat{h}, \hat{a}; h, a) = \begin{cases} 3 & \text{if } (\hat{h}, \hat{a}) = (h, a) \\ 2 & \text{if } \hat{r} = r \text{ and } \hat{h} - \hat{a} = h - a \\ 1 & \text{if } \hat{r} = r \\ 0 & \text{otherwise.} \end{cases}$$

Anchor: `scripts/scoring.py:score_match` — tiers checked in exactly this order (exact, then result+goal-difference, then result).

The tiers are **nested**: an exact hit implies the right goal difference, which implies the right result. So the score decomposes into three indicator variables,

$$S = \mathbf{1}[\hat{r} = r] + \mathbf{1}[\hat{r} = r, \hat{h} - \hat{a} = h - a] + \mathbf{1}[(\hat{h}, \hat{a}) = (h, a)],$$

and taking expectations under the match’s scoreline distribution $P(h, a)$ gives the identity the code actually computes:

$$\mathbb{E}[S(\hat{h}, \hat{a})] = P(\text{result}) + P(\text{result and goal difference}) + P(\text{exact}).$$

Anchor: `scripts/ev321.py:ev_321` — returns $p_{\text{exact}} + p_{\text{rg}} + p_{\text{r}}$, the three terms summed over the scoreline grid $\{0, \dots, 8\}^2$ ($\text{MAX}_G = 8$).

The decision problem. Given a scoreline distribution $P(h, a)$ for the match (built in §3), the model’s pick is the expected-points maximizer

$$(\hat{h}, \hat{a}) = \arg \max_{(\hat{h}, \hat{a}) \in \{0, \dots, 8\}^2} \mathbb{E}[S(\hat{h}, \hat{a}; h, a)],$$

found by brute force over all 81 candidate scorelines.

Anchor: `scripts/ev321.py:best_pick` — exhaustive search over the 9×9 grid; ties keep the first maximizer encountered in row-major order.

Why this is not “pick the most likely result.” The objective rewards margins, not just winners, and that changes the optimum in two systematic ways:

1. *One-goal margins dominate.* For realistic goal rates, the goal-difference tier $P(h - a = 1)$ concentrates on $\hat{h} - \hat{a} = 1$, pulling picks toward 1-0 and 2-1 (made precise in §3).
2. *Draws pool their goal difference.* Every draw shares $h - a = 0$, so a 1-1 prediction banks the 2-point tier on **any** drawn scoreline:

$$\mathbb{E}[S(1, 1)] = 2p_D + P(1, 1),$$

since $\mathbf{1}[\hat{r} = r]$ and the goal-difference indicator coincide whenever the match is drawn. No win prediction enjoys this pooling — a 1-0 pick needs the margin to be exactly one. On evenly-matched games (p_D near its Poisson ceiling), 1-1 is therefore the expected-points optimum even when neither side is a likely winner.

As implemented. `ev321.py` evaluates $\mathbb{E}[S]$ exactly (no simulation) on the truncated grid $\{0, \dots, 8\}^2$; the truncation error is the Poisson tail mass beyond 8 goals, negligible at football rates ($P(X > 8) \approx 2.4 \times 10^{-4}$ per team at $\lambda = 2$, and the rates in play are mostly below that). `scoring.py:score_match` applies the same tiers to realized results during the live run, alongside two diagnostics: the outcome probability assigned to the realized result, and the Brier score $\sum_{j \in \{H, D, A\}} (p_j - \mathbf{1}[j = r])^2$.

1. Inputs — from information to numbers

Everything downstream is arithmetic on numbers, so the first real modeling step is **measurement**: turning four kinds of worldly information into typed numerical objects. This section gives the encoding map for each.

(a) Bookmaker odds $\rightarrow$ probability triples. Each group fixture is one JSON record carrying decimal odds and their implied fair probabilities:

$$\text{quote } (o_H, o_D, o_A) \mapsto (p_H, p_D, p_A), \quad p_j \propto 1/o_j$$

(the normalization is §2's subject). Odds were hand-collected June 2–7, 2026 from bet365, FanDuel, DraftKings, Betfair, oddschecker/oddspeedia aggregators, and sportytrader composites including Kalshi/Polymarket — one source per fixture, recorded in a `source` field (e.g. "FanDuel 2026-06-05"), not averaged across books. The loader merges three files (`odds_AD.json`, `odds_EH.json`, `odds_IL.json`, 24 fixtures each = 72), keys them by the canonicalized team pair, flips home/away orientation if a quote was stored reversed, and **defensively renormalizes** $p_j \leftarrow p_j / (p_H + p_D + p_A)$ so rounding in the stored triples cannot leak through.

Anchor: `scripts/predict_groups.py:load_odds + main` — pair-keyed merge, orientation flip, renormalization.

For matchday-3 fixtures with no published odds at collection time, the probability triple is instead an **Elo estimate** (the §3 logistic applied to the two ratings, flagged in `source`, e.g. "Elo estimate (ARG 2113, AUT 1830)") — the weakest data tier, used for 17 of the 72 group fixtures, mostly on matchday 3.

(b) Elo table $\rightarrow$ ratings R_i . One integer per team for all 48 teams (Spain 2165 down to Qatar 1423), read from `data/elo_outright_news.json` as collected June 5–7, 2026. The effective knockout rating adds two deterministic corrections:

$$R_i^{\text{eff}} = R_i + \delta_i^{\text{inj}} + \delta_i^{\text{host}},$$

with the injury and host terms defined in (c) and §4.

Anchor: `scripts/simulate.py:ko_rating` — `ELO[team] + ADJ.get(team, 0)` plus the host bonus.

(c) Team news → additive rating penalties. Qualitative injury reports are encoded as fixed Elo deductions δ_i^{inj} , set once at collection time:

Team	δ_i^{inj}	Reason on record
Netherlands	−23	Xavi Simons out
Brazil	−20	Rodrygo out
Japan	−10	Mitoma out
Croatia	−10	squad fitness
Spain	−5	Fermín López out (metatarsal)

This is the bluntest measurement in the pipeline — a hand-set scalar per absence, not a player-value model — and is flagged as such per the honesty rule.

Anchor: `data/elo_outright_news.json:injury_elo_adj`, applied in `scripts/simulate.py:ko_rating`.

(d) Tournament structure → fixture list + bracket graph. The 72 group fixtures are a static table (`row`, `group`, `home`, `away`); the knockout is a directed graph encoded as the sheet’s slot layout (16 R32 pairs feeding 8 R16 pairs, 4 QF, 2 SF, final + third-place playoff). Two auxiliary encodings matter:

- **Name canonicalization:** a deterministic alias map ("Türkiye" → "Turkey", "Korea Republic" → "South Korea", ...) so that every data source keys to one team identity — the unglamorous half of information-to-data.
- **Match numbering:** a fixed bijection between sheet rows 4–75 and FIFA’s official group-match numbers 1–72, cross-checked by a unit test, so results logged by match number join correctly to predictions made by row.

Anchor: `scripts/fixtures.py:GROUP_FIXTURES/KNOCKOUT/ALIASES/canon/ROW_MATCH` (numbering guarded by `tests/test_match_numbering.py`).

As implemented. All four inputs are frozen files under `data/` committed before kickoff (locked 2026-06-10, tag `prereg-2026`); nothing in Part I re-reads the outside world. The model’s information set is exactly: one odds quote per fixture, one Elo integer per team, five injury scalars, and the bracket wiring.

2. Clean the odds — de-vigging

A decimal quote o_j implies probability $1/o_j$, but a book’s implied probabilities sum to more than one. The excess is the **overround** (vig),

$$V = \sum_{j \in \{H,D,A\}} \frac{1}{o_j} - 1 > 0,$$

the bookmaker’s built-in margin. The model strips it by **proportional normalization**:

$$p_j = \frac{1/o_j}{\sum_{k \in \{H,D,A\}} 1/o_k}, \quad j \in \{H,D,A\}.$$

Worked example (the one in the companion’s §2): implied 0.50/0.30/0.28 sums to 1.08, so $V = 8\%$; dividing through gives fair probabilities 0.463/0.278/0.259 — the “46/28/26” of the plain-language walkthrough. A real one from the data: Mexico–South Africa at (1.40, 4.60, 8.00) implies (0.714, 0.217, 0.125), $V = 5.6\%$, normalizing to the stored (0.676, 0.206, 0.118).

What proportional de-vigging assumes — honesty note. Dividing by the total spreads the vig across outcomes *in proportion to their implied probability*. That is the standard first-order method, but it is a modeling choice: books are known to shade longshots more than favorites (the favorite–longshot bias), which power or Shin-type de-vigging methods would correct by shrinking small $1/o_j$ relatively more. The pipeline knowingly uses the proportional map — the bias is second-order at World Cup group-match prices, and the downstream pick rule (§3) is insensitive to probability perturbations of that size.

As implemented. De-vigging is applied at data-entry time — each odds record stores both the raw quotes (o_H, o_D, o_A) and the normalized triple (p_H, p_D, p_A), so the transformation is auditable per fixture — and the loader re-applies the same normalization defensively against rounding drift in the stored values.

Anchor: `data/odds_*.json` (fields `odds_*` and `p_*`); `scripts/predict_groups.py:main` — `ph`, `pd`, `pa` = `ph/s`, `pd/s`, `pa/s`.

3. Predict each match — the Poisson scoreline model

The model. Goals are independent Poisson counts:

$$P(h, a) = \text{Pois}(h; \lambda_H) \text{Pois}(a; \lambda_A) = e^{-\lambda_H} \frac{\lambda_H^h}{h!} \cdot e^{-\lambda_A} \frac{\lambda_A^a}{a!},$$

truncated to the grid $(h, a) \in \{0, \dots, 8\}^2$ everywhere.

Anchor: `scripts/poisson_model.py:pois` (`MAX_G = 8`).

Honesty note: real scorelines exhibit mild dependence between the two counts (bivariate-Poisson and Dixon–Coles corrections exist for exactly this); the pipeline uses independence throughout. At World Cup rates the correction mostly reweights low-scoring draws, and the pick rule below conditions on the predicted outcome anyway, which absorbs most of the effect.

The inverse problem. The market gives (p_H, p_D, p_A) ; the model needs rates. Outcome probabilities under the model are the grid sums

$$\tilde{p}_H(\lambda_H, \lambda_A) = \sum_{h>a} P(h, a), \quad \tilde{p}_D = \sum_{h=a} P(h, a), \quad \tilde{p}_A = \sum_{h<a} P(h, a),$$

and `fit_rates` inverts the map by constrained least squares on a lattice:

$$(\hat{\lambda}_H, \hat{\lambda}_A) = \arg \min_{\substack{\lambda_H, \lambda_A \in \{0.10, 0.15, \dots, 3.50\} \\ 1.6 \leq \lambda_H + \lambda_A \leq 3.4}} \sum_{j \in \{H, D, A\}} (\tilde{p}_j(\lambda_H, \lambda_A) - p_j)^2.$$

The lattice step is 0.05; the total-goals band $[1.6, 3.4]$ encodes the prior that World Cup group games average roughly 2.5–2.7 total goals while letting lop-sided matches drift higher. Three targets, two parameters: the system is over-determined and the fit is a projection, not an interpolation.

Anchor: `scripts/poisson_model.py:outcome_probs/fit_rates — steps = 0.05k`, $k = 2..70$; `outcome_probs` is `lru_cached` so the 69×69 grid search costs one pass.

Worked example (Netherlands–Japan, the companion’s §3): fair probabilities $(0.46, 0.26, 0.28)$ fit to $(\hat{\lambda}_{\text{NED}}, \hat{\lambda}_{\text{JPN}}) = (1.5, 1.1)$, under which $P(1, 1) = 12.3\%$ is the modal scoreline but the modal *outcome* is a Netherlands win.

Three pick rules, one published readout. The pipeline distinguishes three rules.

Rule 1 — EV-optimal (§0): $\arg \max \mathbb{E}[S]$ over all 81 scorelines — the theoretical optimum under the pool rule.

Rule 2 — Modal-conditional: most likely scoreline *conditional on the modal outcome*,

$$(\hat{h}, \hat{a}) = \arg \max_{(h, a) : \text{sgn}(h-a) = r^*} P(h, a), \quad r^* = \arg \max_{j \in \{H, D, A\}} p_j,$$

which never predicts a draw for a match it calls a win. Checked exhaustively: it matches rule 1 on **71 of 72** group fixtures.

Anchor: `scripts/poisson_model.py:modal_score`.

Rule 3 — Realistic readout (the published picks, revised pre-kickoff 2026-06-10): round the fitted expected goals while preserving the locked result r^* ,

$$(\hat{h}, \hat{a}) = \begin{cases} (c, c), & c = \text{round}(\frac{\lambda_H + \lambda_A}{2}) \\ (\text{round}(\lambda_H), \text{round}(\lambda_A)) \text{ reordered, winner bumped } +1 \text{ if level} & r^* \in \{H, A\}. \end{cases} \quad r^* = D$$

In knockouts the locked winner takes the higher score; if the rounded scores tie and the winner's rate edge is below 0.25 goals, the pick is a level score decided on penalties.

Anchor: scripts/realistic_scores.py:realistic/ko_realistic — pen threshold (lw - ll) >= 0.25.

The revision is evidence-based, not aesthetic: on the 128 real matches of 2018+2022, the realistic readout lands 18/128 exact scores versus 16 for the EV pick and is about 10% closer in total-goals error, at no measurable pool-points cost (`scripts/backtest.py`). The *model* — probabilities, bracket, champion — is unchanged; only the scoreline readout differs.

Why the optimum lives at 1-0 / 2-1 / 1-1. Under rule 1 the margin tier makes $P(h-a = \hat{h} - \hat{a}, \hat{r} = r)$ the dominant term. For a favorite at World Cup rates, $P(h-a = 1) > P(h-a = m)$ for any $m \geq 2$ (Skellam mass concentrates at small margins), so the optimal winning pick carries margin one, and among margin-one scores the lowest-scoring (1-0) has the largest exact-hit probability since $P(h, a)$ decays in $h + a$. For near-even matches, §0's pooling identity $\mathbb{E}[S(1, 1)] = 2p_D + P(1, 1)$ beats every win pick whose $p_{r^*} \approx p_D$. Netherlands–Japan resolves accordingly: NED favored, so the pick is the modal Netherlands–win score 1-0, not the modal score 1-1.

Knockout probabilities without a market. Undrawn knockout ties have no odds, so (p_H, p_D, p_A) is synthesized from effective Elo ratings ($d = R_H^{\text{eff}} - R_A^{\text{eff}}$, §1b):

$$e = \frac{1}{1 + 10^{-d/400}}, \quad p_D = 0.30 e^{-|d|/700}, \quad p_H = \max(0.01, e^{-\frac{p_D}{2}}), \quad p_A = \max(0.01, 1 - p_H - p_D),$$

renormalized to sum to one, then fed through the same `fit_rates` inversion. The draw share $0.30 e^{-|d|/700}$ is a calibration choice: 30% for dead-even ties, decaying as the mismatch grows. A 100-point Elo gap gives $e \approx 0.64$; 200 points ≈ 0.76 .

Anchor: scripts/realistic_scores.py:main (KO branch); the same logistic appears in scripts/predict_knockout.py:probs (§5).

As implemented. Group rates come from market triples (§2); KO rates from the Elo synthesis above; both pass through the same lattice inversion. All probability arithmetic is exact on the truncated grid — no simulation enters until §4.

4. Simulate the tournament 200,000 times — the Monte Carlo sampler

The estimand. Quantities like “P(Spain champion)” have no closed form — the bracket chains 104 dependent matches through standings, third-place allocation, and knockout routing. They are expectations over tournament realizations ω ,

$$\theta = \mathbb{E}[\mathbf{1}\{\text{event}(\omega)\}] \approx \hat{\theta}_N = \frac{1}{N} \sum_{n=1}^N \mathbf{1}\{\text{event}(\omega_n)\},$$

estimated by drawing N full tournaments. The locked run used $N = 200,000$, so the Monte Carlo standard error at a champion-scale probability is

$$\text{SE}(\hat{\theta}) = \sqrt{\frac{\theta(1-\theta)}{N}} \approx \sqrt{\frac{0.27 \times 0.73}{200,000}} \approx 0.001,$$

i.e. about ± 0.1 percentage points — the “stable, not simulation noise” claim of the companion, made exact.

Anchor: `scripts/simulate.py` — `N_SIMS = int(sys.argv[1])` (default 20,000; the published run passed 200,000), `random.seed(2026)`.

One tournament draw. Each ω_n is generated in four stages:

(i) *Group scorelines.* Every group match draws an exact score from its fitted rates (§3) by inverse-CDF sampling of each truncated Poisson independently:

$$h \sim \text{Pois}(\lambda_H) \wedge 8, \quad a \sim \text{Pois}(\lambda_A) \wedge 8,$$

where the residual tail mass beyond 8 collapses onto 8.

Anchor: `scripts/simulate.py:sample_score` — *cumulative-sum inversion of `pois(k, lam)` over $k = 0..8$.*

(ii) *Standings.* Group tables sort descending by the key

$$(\text{Pts}, \text{GD}, \text{GF}, U), \quad U \sim \text{Unif}(0, 1),$$

with one correction: if two adjacent teams tie exactly on all three stats, their head-to-head result (if decisive) reorders them. *Honesty note:* FIFA’s full tiebreaker cascade includes multi-team head-to-head mini-tables, fair-play points, and drawing of lots; the code implements Pts/GD/GF + pairwise head-to-head + a random tail. The cheap approximation matters only on exact three-stat ties, which carry little probability mass.

Anchor: `scripts/simulate.py:simulate_group` — sort key `(pts, gd, gf, random.random())`, two-way h2h swap.

(iii) *Third-place qualification and slotting.* The 12 third-placed teams rank by the same (Pts, GD, GF, U) key; the top 8 advance. FIFA’s allocation table is encoded as a constraint set per R32 slot — e.g. match 74 may only receive a third from groups $\{A, B, C, D, F\}$ — and the 8 qualifiers are placed by **bipartite matching**, solved by backtracking with the candidate order shuffled so that, when several feasible assignments exist, the simulation samples among them rather than fixing one. If the drawn combination of groups admits no feasible matching, the 8th-ranked third is swapped for the 9th; a still-infeasible draw is discarded.

Anchor: `scripts/simulate.py:THIRD_SLOTS/assign_thirds` and the qualification block in the main loop.

Caveat (known issue, documented 2026-06-10): sampling among feasible third-place assignments is fine for the **pre-tournament** forecast — champion and advancement probabilities are measurably identical under random vs deterministic slotting, because advancement depends on ranking top-8-among-thirds, not on which slot a team fills. It becomes a problem when **conditioning on realized knockout results** (§10): a recorded R32 winner who is a third-placed team is honored only in the iterations that happened to slot that team into that match. A deterministic fix is prepped in `scripts/condition.py`, pending FIFA’s announced bracket (~27 June); see `archive/docs_superpowers/2026-06-10-ko-conditioning-issue.md`.

(iv) *Knockouts.* Each tie is a single Bernoulli draw from the Elo logistic on effective ratings (§1b):

$$P(A \text{ beats } B) = \frac{1}{1 + 10^{-(R_A^{\text{eff}} - R_B^{\text{eff}})/400}}.$$

There is no separate draw state: the logistic is read as the probability of advancing *by any means*, extra time and shootout included (“Elo-weighted shootout”). The host bonus decays in the late rounds: +40 through R32/R16, +20 from the quarter-finals on (crowds matter less in one-off late games at neutral-ish venues).

Anchor: `scripts/simulate.py:ko_rating/ko_winner` — `late=True` for QF/SF/final; bracket wiring in `dicts R32/R16/QF/SF`.

Outputs. Counters over the N draws give champion / finalist / semi-finalist distributions, per-matchup materialization rates for the sheet’s predicted bracket (“how often does Germany–France actually happen at match 89”), and the per-slot winner distribution that §5 turns into picks. A second pass with the RNG reseeded to 2026 recomputes the slot-winner counters so both passes see identical group-stage randomness.

As implemented. Plain `random` (Mersenne Twister), single seed 2026, no variance reduction — at $N = 200,000$ none is needed. The fitted rates are computed once per fixture before the loop (the `lru_cache` on `outcome_probs` makes the lattice inversion a one-time cost), so a draw is 72 score samples plus ≤ 32 Bernoulli draws.

5. Optimize the bracket — slot emergence as a formula

Why “stronger team” is the wrong criterion. A knockout pick at bracket slot m scores only if the named team **occupies** the slot *and wins* it. For team t and slot m , define from the §4 simulation

$$E_m(t) = P(t \text{ wins match } m) = P(t \text{ reaches } m) \cdot P(t \text{ wins} \mid t \text{ reaches } m),$$

the **slot emergence** of t at m — estimated as the fraction of the N simulated tournaments in which t is the winner recorded at slot m . The pick rule is simply

$$\hat{t}_m = \arg \max_t E_m(t).$$

A strong team that splits its qualification between first and second place spreads its mass across *different* slots; a weaker team that reliably finishes second concentrates its mass in one. The argmax sees the product, not the strength — this is the criterion that overturned three naive picks on the sheet.

Anchor: `scripts/simulate.py` (second pass, `slot_win` counters; the `FLIP?` flag prints whenever the sheet’s opponent out-emerges the sheet’s pick) and `scripts/slot_value.py:simulate/entries`.

Honesty note: the optimization is **greedy per slot**, not a joint argmax over path-consistent brackets — each slot takes its own maximizer, and the author keeps the sheet path-consistent by hand. A full joint optimization over the 2^{31} advancement patterns was not attempted; the per-slot criterion already captures the occupancy–strength tradeoff that drives the value.

Quantifying the criterion’s value. `slot_value.py` compares three entry strategies over all 31 knockout matches, scoring each by its expected knockout points $\sum_m E_m(\hat{t}_m)$ (under the pool rule a knockout pick’s expected points are proportional to exactly this win-the-slot probability):

1. *Slot-emergence:* $\hat{t}_m = \arg \max_t E_m(t)$,
2. *Modal occupant:* the team most often **reaching** m (occupancy without the win term),
3. *Strongest reachable:* the highest-rated team that ever reaches m .

The comparison is repeated on the actual 2026 bracket and on four strength-perturbed replicas (independent $\mathcal{N}(0, 80^2)$ noise added to every Elo rating,

group probabilities rebuilt from the perturbed ratings), so the conclusion — emergence beats both rivals — is a property of the criterion, not of one bracket draw. The study (like `condition.py` and the other analysis-layer scripts) also carries a **market-implied Elo recalibration** of the Portugal/Colombia pair: the market priced their head-to-head as a 140-point gap where raw Elo showed 7, so the difference is split as Portugal +66, Colombia −67. Adopted at a prediction refresh, this is part of the knockout machinery from `condition.py` onward but absent from the original `simulate.py` constants — worth knowing when reconciling numbers across scripts.

Anchor: `scripts/slot_value.py` — $N = 20,000$ per bracket, perturbation seeds $100+k$, simulation seed 2026.

Attaching scorelines to knockout picks. Each predicted tie gets a score via the same machinery as §3: Elo-synthesized (p_H, p_D, p_A) (the §3 logistic with decaying draw share), `fit_rates` inversion, modal-conditional readout. If the modal outcome is a draw, the full-time pick stays level and the higher-rated side gets +1 in the OT/PK column — the sheet’s encoding of “wins on penalties.”

Anchor: `scripts/predict_knockout.py:probs/predict_pair`.

Pivotality — which single results carry the forecast. For an upcoming match m with pre-match champion distribution p^{pre} , fork the match across its possible outcomes o (three for a group match, weighted by the §2 fair probabilities; two for a knockout, weighted by the Elo win expectancy), recompute the conditioned champion distribution $p^{\text{post}(o)}$, and average the information distance:

$$\text{Piv}(m) = \sum_o w_o D_{\text{KL}}(p^{\text{post}(o)} \| p^{\text{pre}}) \quad \text{bits},$$

with an $\varepsilon = 10^{-6}$ floor added to both distributions before the $\log_2$ ratio. This is the *expected* forecast swing of playing match m — the ex-ante counterpart of the realized per-game KL the Prophet tracks in §11. The headline finding: a knockout match is several times as pivotal as a typical group match, and the most pivotal calls cluster on the predicted champion’s path.

Anchor: `scripts/pivotality.py:kl/main` — group outcomes encoded as scores $(1,0)/(1,1)/(0,1)$, $N = 5,000$ per fork, seed 2026, output `data/pivotality.json`.

As implemented. Slot emergence and pivotality are decision *diagnostics* — they choose and stress-test the sheet’s knockout entries; they do not feed back into the match-level probabilities, which remain market- and Elo-driven (§§2–3).

6. Check the risk — the machine learning layer

The supervised problem. The simulator is also a data generator. Each simulated tournament yields one labeled example of the question “given which

group calls came true, how many knockout picks scored?”:

- **Features** $x \in \{0, 1\}^{24}$: for each group $g \in \{A, \dots, L\}$, two indicators — *winner correct* ($\mathbf{1}[\text{simulated group winner} = \text{sheet's predicted winner}]$) and *runner-up correct* (same for second place).
- **Target** $y \in \{0, \dots, 31\}$: the number of the sheet’s 31 knockout advancers ($16 + 8 + 4 + 2 + 1$) that advance in that simulation.

The dataset is $N = 40,000$ draws (seed 7), split 75/25 train/test, and two regressors $\hat{f}(x) \approx \mathbb{E}[y \mid x]$ are fit side by side:

Model	Hyperparameters
Random Forest	300 trees, max depth 12, min leaf 20
XGBoost	400 trees, max depth 4, learning rate 0.05, subsample 0.8, colsample 0.8

Anchor: `scripts/ml_risk_xgb.py` (XGBoost + SHAP), `scripts/ml_risk.py` (Random Forest original); both use the same surrogate generator and the market-implied Portugal/Colombia recalibration of §5.

What is measured.

1. *Explained variance.* The test R^2 of $\hat{f}$ answers “how much of the knockout score is determined by the group stage at all?” — about half, the companion’s “group-stage correctness explains only ~50%; the rest is knockout luck.” The agreement of two different model families (and a Spearman rank correlation of their importance vectors) certifies this is not an artifact of one learner.
2. *Leverage per call.* SHAP values decompose each prediction additively, $\hat{f}(x) = \phi_0 + \sum_{i=1}^{24} \phi_i(x)$; the report ranks features by mean $|\phi_i|$ and quantifies the directional effect $\mathbb{E}[\phi_i \mid x_i = 1]$ — expected knockout points gained when group call i comes true. The top of the ranking (“Spain wins Group H”, roughly +0.8 points, about $3\times$ the weakest group) is the model’s own risk concentration made explicit: Spain’s predicted path runs six matches deep, so that one group call props up the whole champion run.

Honesty note — what this layer is and is not. The regression is trained on the simulator’s *own* output: it is a **sensitivity analysis of the model**, a variance-decomposition diagnostic, not an independent second forecast and not a validation against reality. It changes no pick; its product is the ranked list of calls to protect.

Anchor: `scripts/ml_risk_xgb.py` — `shap.TreeExplainer`, output `data/ml_risk_xgb.json`.

Part II — Avraa’s Prophet: the self-learning forecaster

Part I’s forecast is frozen by design. The Prophet asks the live question: *as the tournament speaks, how should the forecast listen?* It runs two tracks side by side — the **Frozen Prophet** (control: re-condition on results as they arrive, team strengths fixed at their locked values) and the **Learning Prophet** (treatment: strengths drift in response to match *performance*, not just results). The headline quantities are the gap between the two tracks’ champion trajectories and the per-game information ledger of §11 — which matches actually taught the model something. Hard constraint inherited from the project: **free data sources only**.

7. Measurement: from match events to λ_{obs}

The signal problem. Learning from results alone is slow — 1-0 carries one bit of winner information but little about *how* the teams played. The natural performance signal is expected goals (xG), but real xG is not freely available for World Cups (Understat covers the top-5 European leagues only). The free signal that survives is the box score: shots on target and other shots.

The proxy. Define the observed performance rate of a team in a match as

$$\hat{\lambda}_{obs} = \max(0, \beta_1 \cdot \text{SoT} + \beta_2 \cdot \text{otherShots}), \quad \beta_1 = 0.3256, \quad \beta_2 = 0,$$

where otherShots = off-target + blocked. The calibrated β_2 is exactly zero (see below), so in practice the proxy reads: **each shot on target is worth about a third of an expected goal**.

Anchor: scripts/performance.py:proxy_xg; coefficients in data/proxy_coef.json — sot: 0.32557, other: 0.0. Box-score parsing (other_shots = off_target + blocked) in scripts/collect_match.py:parse_statistics.

Calibration. The coefficients are a one-time least-squares fit of realized goals on shot counts over the 2018 and 2022 World Cups — $n = 236$ team-matches of free ESPN box-score data:

$$(\beta_1, \beta_2) = \arg \min_{\beta \geq 0} \sum_{i=1}^{236} (\text{goals}_i - \beta_1 \text{SoT}_i - \beta_2 \text{otherShots}_i)^2,$$

OLS through the origin (no intercept: zero shots should mean zero expected goals), solved in closed form and clipped to non-negative coefficients. The clip is what zeroes β_2 : off-target and blocked shots carry no incremental goal information once shots on target are counted.

Anchor: `scripts/calibrate_proxy.py:fit — np.linalg.lstsq` on the 236×2 design, `SEASONS = world-cup-2018, world-cup-2022`.

Validation (full design in §12): the coefficient is stable across tournaments (refit on 2018 alone vs 2022 alone brackets the locked 0.326), the cross-tournament goal RMSE beats the naive predict-the-mean baseline in both directions, and a placebo test confirms the proxy carries real signal. The honest reading stands: this is a *poor man's xG* — it ignores shot quality, location, and game state — but it is free, it is available within minutes of full time, and it beats not measuring performance at all.

The fallback. If a real xG number is available for a match, it takes precedence:

$$\lambda_{obs} = \begin{cases} \text{xG}^{\text{real}} & \text{if supplied} \\ \hat{\lambda}_{obs}(\text{SoT, otherShots}) & \text{otherwise,} \end{cases}$$

with the source recorded per match so the two never mix silently.

Anchor: `scripts/performance.py:compute_lambda_obs — source: 'real' / 'proxy'`.

8. The expectation model: λ_{exp} from ratings

To be surprised, the model first needs an expectation. Given the two teams' current ratings, the expected performance rates are produced by the *same* chain Part I uses for odds-free matchups — one composition of three maps, applied to the rating difference $d = R_{home} - R_{away}$:

$$d \xrightarrow[\text{Elo logistic + draw}]{(i)} (p_H, p_D, p_A) \xrightarrow[\text{fit_rates}]{(ii)} (\lambda_{exp}^{home}, \lambda_{exp}^{away}),$$

where step (i) is §3's synthesis

$$e = \frac{1}{1 + 10^{-d/400}}, \quad p_D = 0.30 e^{-|d|/700}, \quad p_H = \max(0.01, e^{-\frac{p_D}{2}}), \quad p_A = \max(0.01, 1 - p_H - p_D),$$

(renormalized) and step (ii) is the lattice least-squares inversion of §3. Nothing new is fitted here — the Prophet inherits Part I's match model wholesale, which is what makes the frozen and learning tracks comparable: they disagree only about the ratings fed in, never about the map from ratings to goals.

Anchor: `scripts/learn.py:_lambda_for_diff/lambda_expected`.

Numerics. λ_{exp} depends only on the difference d , rounded to the nearest integer Elo point, and `_lambda_for_diff` is memoized on that integer (`lru_cache`). The rounding is a pure-performance approximation: a half-point rating change

moves the win expectancy by under 10^{-3} , far below the lattice resolution (0.05 goals) of the inversion it feeds, so the cache is lossless in effect. The payoff is large — `fit_rates` is a grid search invoked inside every Monte Carlo iteration, and memoization cut the full replay test suite from minutes to about a second.

9. The learning update: net surprise and regularized drift

Net surprise. After a match, each team’s performance is compared to expectation on both sides of the ball and netted:

$$s_{home} = \underbrace{(\lambda_{obs}^{home} - \lambda_{exp}^{home})}_{\text{attack: out-produced expectation}} - \underbrace{(\lambda_{obs}^{away} - \lambda_{exp}^{away})}_{\text{defense: leaked above expectation}}, \quad s_{away} = -s_{home}.$$

The construction is **zero-sum by definition** — one team’s positive surprise is exactly the other’s negative — so learning redistributes strength rather than inflating it. The expectations λ_{exp} are computed at the learning track’s *current* ratings (baseline + drift), so a team that has already been upgraded must keep over-performing to keep earning.

Anchor: `scripts/learn.py:net_surprise` and `LearningTrack.apply_match`.

The drift update. Each team carries an accumulated drift d_i (initialized to 0) on top of its frozen baseline, $R_i^{\text{learn}} = R_i + d_i$. One match updates it as

$$d_i \leftarrow \gamma d_i + \text{clip}(k s_i, -B, +B), \quad k = 50, \quad \gamma = 0.95, \quad B = 75.$$

Note the exact form, extracted from the code: the **clip binds the per-match step** ks_i , not the accumulated drift. The drift itself is bounded only geometrically — after n matches $|d_i| \leq B \frac{1-\gamma^n}{1-\gamma}$ (about 214 Elo over a 3-game group stage, 453 over a full 7-game run) — though realized drifts in the 2022 replay stayed far inside that envelope.

Anchor: `scripts/learn.py:update_drift` — `step = max(-bound, min(bound, k*surprise)); return decay*drift + step; defaults in LearningTrack.__init__ (k=50.0, decay=0.95, bound=75.0).`

What kind of filter this is. The update is a regularized online estimator of a slowly-varying strength state — exponential forgetting plus a gain on the innovation, the same family as exponentially-weighted Elo variants and a steady-state (fixed-gain) Kalman/EKF update:

- $\gamma = 0.95$ is the **forgetting factor**: absent new surprises, drift decays toward the pre-tournament baseline with half-life $\ln 2 / \ln(1/\gamma) \approx 13.5$ matches — longer than any team’s tournament, so within one World Cup the decay mostly tempers, rather than erases, what was learned.

- $k = 50$ is the **gain**: one full goal of net surprise moves a rating by 50 Elo points (about 7 percentage points of win expectancy at even ratings). Chosen by the two-tournament sweep of §12 (2022 optimum 40, 2018 optimum 60, locked at the midpoint).
- $B = 75$ is **hard regularization** against single-match outliers: no one result — however lopsided the shot count — can move a team more than 75 points.

Setting $k = 0$ makes the step vanish and the drift stay at zero: **the frozen track is exactly the learning track at zero gain**, which is what makes the A/B comparison clean.

Anchor: `scripts/learn.py:LearningTrack` — “ k is the swept knob ($k=0 \Rightarrow$ frozen track)”.

10. Re-simulation and the two-track trajectory

The conditional estimand. After t games have been played, the forecast is no longer §4’s prior — it is the conditional distribution

$$p_t(\text{team}) = P(\text{team wins} \mid \text{results}_{1..t}, \text{ratings}_t),$$

estimated by Monte Carlo with the played games **pinned** and the future sampled: a group fixture found in the results set contributes its realized scoreline to the standings; an unplayed one is sampled from $\text{Pois}(\lambda_{exp})$ at the current ratings; a played knockout tie advances its recorded winner; an unplayed one is a Bernoulli draw from the Elo logistic. The two tracks differ **only** in ratings_t : the frozen track always passes the locked baseline R_i , the learning track passes $R_i + d_{i,t}$ with the drift of §9.

Anchor: `scripts/sim_tournament.py:group_standings/simulate_knockout/simulate` — *results dicts keyed by team pair; win_prob is the Elo logistic; knockout folded as a depth map (entrant depth 1, champion $1 + \log_2(\text{bracket size})$).*

The replay loop. `run_replay` steps an ordered game list. For each finished game it (i) pins the result, (ii) applies the §9 drift update from the game’s λ_{obs} , and (iii) re-simulates the whole tournament **twice** — once per track — recording a snapshot $(p_t^{\text{frozen}}, p_t^{\text{learn}})$ plus the per-game KL of §11. Snapshot $t = 0$ (index -1) is the unconditioned baseline, where the two tracks coincide by construction.

$$\text{trajectory} = \left\{ (p_t^{\text{frozen}}, p_t^{\text{learn}}) \right\}_{t=0}^T, \quad \text{headline} = \text{the gap between the two paths.}$$

Anchor: `scripts/replay.py:run_replay` — $N = 4,000$ per snapshot, seed 2026, $k = 50$ default.

Numerics. Every snapshot reuses the same seed, so consecutive distributions are coupled through common random numbers: the difference between p_t and p_{t-1} reflects the new information (and the rating drift), not resampling noise. At $N = 4,000$ the per-probability MC noise is ≈ 0.7 percentage points at champion scale — coarse for headlines, fine for a 100-snapshot trajectory.

Two engines, honestly distinguished. The format-agnostic engine above runs the learning study (and the 2022 replay validation); it seeds the bracket directly from group positions and ranks groups by a *deterministic* (Pts, GD, GF, stable-order) sort — no third-place machinery, no random tiebreak. The **live 2026 trajectory** instead runs through the production conditioning engine, which carries the full 2026 bracket: third-place ranking, FIFA slot constraints, and the market-implied recalibration of §5.

Anchor: `scripts/condition.py:conditional_probs` (2026-hardcoded; the engine behind `scripts/pivotality.py`, `scripts/record_update.py`, and the live updates).

Caveat (same issue as §4): in the production engine, a recorded R32 winner who qualified as a **third-placed team** is honored only in the Monte Carlo iterations that slotted that team into that match — measured acceptance as low as 10% — so the conditioned trajectory under-responds to those eight possible results. A deterministic third-place assignment is prepped in `condition.py` behind the `REALIZED_THIRDS` table, to be pinned from FIFA’s announced bracket (~27 June) and verified by `tests/test_ko_conditioning.py`; it is baseline-neutral by measurement. Until then, knockout conditioning for third-place winners carries this documented bias. Reference: `archive/docs_superpowers/2026-06-10-ko-conditioning-issue.md`.

11. Information accounting: KL divergence in bits

The per-game ledger. Each snapshot records how far the new champion distribution moved from the previous one, in bits:

$$D_{\text{KL}}(p_t \| p_{t-1}) = \sum_{i: p_t(i) > 0} p_t(i) \log_2 \frac{p_t(i)}{\max(p_{t-1}(i), \varepsilon)}, \quad \varepsilon = 10^{-12}.$$

The direction is *new given old* — “how surprising is today’s belief from yesterday’s vantage.” Terms with $p_t(i) = 0$ contribute nothing (the correct $0 \log 0 = 0$ limit); only the denominator is floored, so a team rising from (near-)elimination registers as a large surprise. Base 2 makes the unit **bits**: one bit is the information in one fair coin flip about the championship.

Anchor: `scripts/snapshot.py:kl_divergence`; attached per snapshot as `kl_from_prev` in `scripts/replay.py:_snapshot`.

This is the *realized* counterpart of §5’s pivotality: pivotality is the expected KL of a match before kickoff; `kl_from_prev` is what the match actually deliv-

ered. Summing the ledger over any window gives “how much the tournament taught the forecast” in that window. In the 2022 replay validation the ledger is sharply peaked: Japan’s two upsets carried 0.13 and 0.09 bits against a median group game of 0.026 — a handful of matches carry most of the tournament’s information.

Channel decomposition — result vs performance. A game informs the forecast through two separable channels:

- **Result channel:** the frozen track’s own per-game divergence, summed over the window — what conditioning on scorelines alone teaches. Over the 48 group games of the 2022 replay: $\sum_t D_{\text{KL}}(p_t^{\text{frozen}} \| p_{t-1}^{\text{frozen}}) = 1.52$ bits.
- **Performance channel:** the divergence **between the tracks** once the window has been played, $D_{\text{KL}}(p_T^{\text{learn}} \| p_T^{\text{frozen}}) = 0.26$ bits — what reading the shot counts added beyond the scorelines, about one part in six.

Across the 48 games the per-game result and performance informations correlate at -0.09 — essentially orthogonal: *how well a team played is largely independent of whether it won*. That near-orthogonality is the empirical license for the learning track’s existence; if the proxy merely relabeled scorelines, the second channel would vanish into the first.

Anchor: computed from `run_replay` trajectories; reported in the paper’s §4.9 (learning study) and `data/replay_2022_trajectory.json`.

12. How we know it works: held-out validation, placebo, k-sweep

The learning claim was stress-tested against the two complete free-data tournaments available, 2018 and 2022, with three designs. The evaluation metric throughout is **finalist lift**: replay a tournament’s group stage, then compare how much champion-probability mass each track’s end-of-groups forecast assigns to the two teams that actually reached that final,

$$\text{lift} = \left(\sum_{t \in \text{finalists}} p_T^{\text{learn}}(t) \right) - \left(\sum_{t \in \text{finalists}} p_T^{\text{frozen}}(t) \right),$$

in percentage points, alongside the champion log-loss $-\ln p_T(\text{actual champion})$.

(1) Leave-one-tournament-out cross-validation. Tune k on one World Cup, test on the other — genuinely out-of-sample in the only unit that matters here, whole tournaments: train-2018 fixes $k = 60$, tested on 2022; train-2022 fixes $k = 40$, tested on 2018. Learning helps out-of-sample in both directions (e.g. +7.2 points of finalist mass carrying the 2018-tuned k into 2022), with the champion log-loss improving in tandem.

Anchor: `scripts/heldout_validation.py` part (1) — $N = 4,000$, seed 7.

(2) Placebo control. If the lift is *signal*, destroying the signal must destroy the lift. Shuffle the λ_{obs} values across the tournament’s games (4 independent shuffles, seeds 0–3) and re-run at the locked $k = 50$: the real-xG lift (about +6.8 points on 2022) collapses to negative territory under shuffling (mean ≈ -5.3) — randomized performance data actively *hurts*, as it should if the proxy carries real per-team information rather than a free-parameter artifact.

Anchor: scripts/heldout_validation.py part (2) — rng.shuffle(lams), N = 3,000 per run.

(3) Proxy stability and calibration. The §7 coefficient refit on each tournament alone brackets the locked value (β_1 fit on 2018 only and on 2022 only straddle 0.326); the cross-tournament goal RMSE (fit on one, predict the other) beats the **naive baseline** (predict every team-match with the tournament’s mean goals) in both tournaments. The proxy is crude, but it is stable and it earns its keep against the cheapest alternative.

Anchor: scripts/heldout_validation.py part (3) — per-tournament refit $c = \sum s_i g_i / \sum s_i^2$ (OLS through the origin), RMSE comparisons.

The k-sweep. The gain was chosen by sweeping $k \in \{0, 20, 40, 60, 80, 120, 160, 220, 300\}$ on full two-track replays of both tournaments, scoring finalist mass and log-loss, and tracking responsiveness as the total-variation distance between the tracks’ final forecasts. The curve is single-peaked in both years — learning helps, then over-reacts: at high k the forecast chases every shot count and the finalist mass falls back below the frozen baseline. Optima: $k = 40$ (2022), $k = 60$ (2018); locked default: the midpoint $k = 50$ (with $\gamma = 0.95$, $B = 75$ fixed throughout).

Anchor: scripts/sweep_k.py (N = 3,000, seed 2022), scripts/run_2018_ksweep.py, figure paper/figs/fig_ksweep_2tourn.pdf.

Limitations, stated plainly. The sample is $n = 2$ tournaments; every number above carries that caveat. The validation shows the learning signal is real and correctly signed out-of-sample — it does not pin the optimal gain to better than “somewhere around 50,” and the study is framed as exploratory in the paper accordingly. The 2026 live run is the genuine test: the locked $k = 50$ meets data it has never seen.